

# Funktionales Sortieren von strukturieren Daten

## Eigene Anwendung

### Einleitung:

---

Sie wenden in einem **eigenen individuellen Beispiel** das Funktionale-Sortieren von strukturieren Daten an.

Die Aufgabe fließt in Ihre Bewertung mit ein.

### Ziel

---

- ⇒ Sie können ein geeignetes Datenbeispiel entwerfen.
- ⇒ Sie können das Datenbeispiel implementieren.
- ⇒ Sie können Testdaten für das Beispiel generieren.
- ⇒ Sie wenden verschiedenste Aspekte des funktionalen Sortierens an.

### Inhalte

---

|                                 |   |
|---------------------------------|---|
| Einleitung: .....               | 1 |
| Ziel .....                      | 1 |
| Inhalte .....                   | 1 |
| Aufgabe: Eigene Anwendung ..... | 2 |
| Bewertung: .....                | 4 |

## Aufgabe: Eigene Anwendung

---

### Minimale Anforderung

Entwerfen Sie eine Datenklasse:

- Mit mindestens 5 verschiedenen Attributen.
- Mit mindestens 4 verschiedenen Datentypen.  
(2 *Datentypen* sind keine primitiven *Datentypen* oder *Wrapperklassen*)

Generieren Sie für Ihre Klasse Datensätze

- Mindestens 100 zufällig generierte Datensätze
- Ein möglicher Datengenerator ist <https://generatedata.com/>  
Passen Sie die generierten Daten so an, dass Sie sie in Java verwenden können.

Eine Alternative wäre Instancio <https://www.instancio.org/>.

Hier eine kurze Erklärung zur Anwendung:

<https://www.instancio.org/user-guide/#instancio-api>

oder auch

<https://www.baeldung.com/java-test-data-instancio>

**Bemerkung:** Werden Testdaten immer wieder neu generiert, dann ist die eindeutige Reproduzierbarkeit je nach Anwendung schwierig.

---

Nun wenden Sie verschiedene Aspekte des funktionalen Sortierens an Ihre Daten an.  
Sie zeigen dabei, dass Sie die unterschiedlichen Aspekte unterscheiden und anwenden können.  
Einige sind aufwendiger, andere kompakt.

Machen Sie das so, dass es gut unterscheidbar ist!

Wenden Sie Ihre Teile immer auch auf Ihre Daten an.

Wenden Sie mindestens an.

Teil 1

- Comparator und Comparable (vom Comparator abgeleitete Klasse)
- Anonyme Klasse
- Lambda Expression
- Comparator Chain

---

Teil 2

- Verschiedene Sortierungen, so dass alle Attribute mindestens 1x verwendet wurden.
  - Mehrstufige Sortierungen
  - Natural Order
  - Reverse Order
-

---

**Erweiterte Anforderung** (freiwillig für eine bessere Bewertung)

Entwerfen Sie eine Datenstruktur von zwei Datenklassen, die über eine Assoziation verbunden sind, z.B. Person und Hund.

Wenden Sie die verschiedenen Aspekte des funktionalen Sortierens auf diese Datenstruktur an. Und dies insbesondere in Bezug auf die Assoziation.

---

**Erweiterte Anforderung** (freiwillig für eine bessere Bewertung, schwer)

Bei dieser Aufgabe tauchen Sie in Java ein. Untersuchen wie etwas detailliert funktioniert und finden einen Weg, dass für eine Leser verständlich zu erklären und zu dokumentieren.

Tauchen Sie in Java ein. Wie funktioniert das genau? Was macht Java?

- Zeigen Sie auf, wie Java die Anwendung der Natural-Order einer Klasse beim Sortieren anwendet.
  - Zeigen Sie auf, wie Java die Reverse-Order an eine Klasse anwendet.
  - Tauchen Sie ein in das Sortieren von Java. Wie sortiert Java?
  - Zeigen Sie auf, wie Java das Sortieren löst.
-

**Bewertung:**

Name: ..

|                                                                                                                                                                                                                                                  |                               |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------|
| <b>Projekt und Daten</b> <ul style="list-style-type: none"> <li>- Datenklasse mit 5 Attributen, 4 versch. Datentypen (2 davon sind keine primitiven Datentypen und keine Wrapperklassen)</li> <li>- 100 Datensätze</li> </ul>                    | genügend, gut<br><br>_v. 4 P. |
| <b>Anwendung Sortieren Teil 1</b> <ul style="list-style-type: none"> <li>- Comparator abgeleitete Klasse</li> <li>- Anonyme Klasse</li> <li>- Lambda Expression</li> <li>- Comparator Chain</li> </ul>                                           | genügend, gut<br><br>_v. 4 P. |
| <b>Anwendung Sortieren Teil 2</b> <ul style="list-style-type: none"> <li>- Verschiedene Sortierungen, so dass alle Attribute 1x verwendet wurden.</li> <li>- Mehrstufige Sortierung</li> <li>- Natural Order</li> <li>- Reverse Order</li> </ul> | genügend, gut<br><br>_v. 4 P. |
| <b>Erweiterte Anforderung</b> <ul style="list-style-type: none"> <li>- Datenstruktur von zwei Datenklassen</li> <li>- Sortieren mit Bezug auf Assoziation</li> </ul>                                                                             | genügend, gut<br><br>_v. 1 P. |
| <b>Erweiterte Anforderung</b> <ul style="list-style-type: none"> <li>- Tiefes Eintauchen</li> <li>- Natural-Order</li> <li>- Reverse-Order</li> <li>- Sortierung</li> </ul>                                                                      | genügend, gut<br><br>_v. 2 P. |

\_ v. 15 P.      Note: -